Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 8

Disciplina: Tehnici de programare

Tema: ” Colecții: liste, tupluri, mulțimi, dicționare”

A efectuat : st. gr.SI – 263 Morozovschii Carolina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Capitolul 1. Scheletul și prima listă (15 min)

Creează arsenal/privesc/enum.py. Întâi citești fișierele SUID într-o listă și o numeri. Un fișier de citire comun îți scoate liniile curate (fără goluri, fără comentarii):

```python
import argparse
import json
import os

GTFO = {"find", "vim.basic", "vim", "nmap", "cp", "bash",
        "less", "awk", "python3", "perl"}


def citeste_linii(cale):
    with open(cale, encoding="utf-8") as f:
        return [
            linie.strip()
            for linie in f
            if linie.strip() and not linie.lstrip().startswith("#")
        ]

```

Fig.1.1-Codul sursă (enum.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborato
r_Colectii_start\laborator_colectii\arsenal\privesc> python enum
.py
linii SUID: 14
```

Fig.1.2-Rezultatul codului (enum.py)

Capitolul 2. Nume unice: de la listă la mulțime (12 min)

Numele fișierului e ce decide dacă e periculos (find, vim, nmap...). Scoate numele din fiecare cale și ține-le o singură dată.

```python
def nume(cale):
    return cale.rsplit("/", 1)[-1]
```

Fig.2.1-Codul sursă (enum.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborato
r_Colectii_start\laborator_colectii\arsenal\privesc>     python
enum.py
linii SUID: 14
SUID distincte: 13
```

Fig.2.2- Rezultatul codului (enum.py)

Capitolul 3. Constatările SUID: listă de dicționare (18 min)

Fiecare SUID devine o fișă: calea, numele, riscul. Periculos = numele e într-o mulțime de binare cunoscute ca scăpări (stil GTFOBins). Întorci o listă de dicționare, sortată, fără dubluri.

```python
def suid_constatari(cale):
    cai = citeste_linii(cale)
    return [
        {
            "cale": c,
            "nume": nume(c),
            "risc": "mare" if nume(c) in GTFO else "normal",
        }
        for c in sorted(set(cai))
    ]
```

Fig.3.1- Codul sursă (enum.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborato
r_Colectii_start\laborator_colectii\arsenal\privesc> python enum
.py
linii SUID: 14
SUID distincte: 13
13 SUID, 4 cu risc mare
['cp', 'find', 'nmap', 'vim.basic']
```

Fig.3.2- Rezultatul codului (enum.py)

Capitolul 4. Sudo și cron: tuplul și gruparea (20 min)
O linie din sudo.txt are forma „user : drept”. O despachetezi într-un tuplu. O linie din crontab.txt are șase câmpuri de program, apoi userul și comanda. Fiecare sursă dă o listă de dicționare.

```python
def sudo_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        user, drept = ln.split(" : ", 1)
        drept = drept.strip()
        out.append({
            "user": user.strip(),
            "drept": drept,
            "risc": "mare" if "NOPASSWD" in drept else "normal",
        })
    return out


def cron_constatari(cale):
    out = []
    for ln in citeste_linii(cale):
        campuri = ln.split()
        if len(campuri) >= 7:
            out.append({
                "user": campuri[5],
                "comanda": " ".join(campuri[6:]),
            })
    return out
```

Fig.4.1- Codul sursă (enum.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_start\laborator_colectii\arsenal\privesc> python enum.py s
tart
Traceback (most recent call last):
  File "C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_start\laborator_colectii\arsenal\privesc\enum.py", li
ne 1, in <module>
    import argparse
  File "C:\Users\moroz\AppData\Local\Programs\Python\Python315\Lib\argparse.py", line 90, in <module>
    import re as _re
  File "C:\Users\moroz\AppData\Local\Programs\Python\Python315\Lib\re\__init__.py", line 126, in <module>
    import enum
  File "C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_start\laborator_colectii\arsenal\privesc\enum.py", li
ne 2, in <module>
    import json
  File "C:\Users\moroz\AppData\Local\Programs\Python\Python315\Lib\json\__init__.py", line 105, in <module>
    from .decoder import JSONDecoder, JSONDecodeError
  File "C:\Users\moroz\AppData\Local\Programs\Python\Python315\Lib\json\decoder.py", line 5, in <module>
    from json import scanner
  File "C:\Users\moroz\AppData\Local\Programs\Python\Python315\Lib\json\scanner.py", line 11, in <module>
    NUMBER_RE = re.compile(
AttributeError: partially initialized module 're' from 'C:\Users\moroz\AppData\Local\Programs\Python\Python315\Lib\re\__init__.py' ha
s no attribute 'compile' (most likely due to a circular import)
```

Fig.4.2- Rezultatul codului (enum.py)

Capitolul 5. Raportul: JSON și rezumat pe categorii (15 min)
main() cu argparse: primește folderul capturilor și un comutator --json. Cu --json scoate JSON curat; fără el, un rezumat pe categorii (cu .items()) și salvează raportul pe disc.

```python
def enumereaza(d):
    return {
        "suid": suid_constatari(os.path.join(d, "suid.txt")),
        "sudo": sudo_constatari(os.path.join(d, "sudo.txt")),
        "cron": cron_constatari(os.path.join(d, "crontab.txt")),
    }


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("capturi")
    parser.add_argument("--json", action="store_true")
    args = parser.parse_args()

    raport = enumereaza(args.capturi)
    if args.json:
        print(json.dumps(raport, indent=2, ensure_ascii=False))
        return

    for categorie, lista in raport.items():
        mari = [c for c in lista if c.get("risc") == "mare"]
        print(f"{categorie}: {len(lista)} gasite, {len(mari)} cu risc mare")

    utilizatori = {c["user"] for c in raport["sudo"]} | {
        c["user"] for c in raport["cron"]
    }
    print("utilizatori implicati:", ", ".join(sorted(utilizatori)))

    with open("enum.json", "w", encoding="utf-8") as f:
        json.dump(raport, f, indent=2, ensure_ascii=False)
    print("Raport salvat in enum.json")


if __name__ == "__main__":
    main()
```

Fig.5.1- Codul sursă (enum.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_start\laborator_colectii> python -m arsenal.privesc.enum a
rsenal/privesc/start
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

Fig.5.2-Rezultatul codului (enum.py)

Capitolul 6. Citirea căilor în mulțimi
Sarcina: Să citesc căile din fișierul de bază și din captura curentă în două mulțimi.

```python
from arsenal.privesc.enum import GTFO, citeste_linii, nume

def cai(fisier):
    return set(citeste_linii(fisier))


baza = cai("arsenal/privesc/start/baza_suid.txt")
acum = cai("arsenal/privesc/start/suid.txt")
print("baza:", len(baza), " acum:", len(acum))
```

Fig.6.1- Codul sursă (diferenta.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_st
art\laborator_colectii> python -m arsenal.privesc.diferenta
baza: 9  acum: 13
```

Fig.6.2-Rezultatul codului (diferenta.py)

Capitolul 7. Găsirea fișierelor noi
Sarcina: Să scad mulțimea de bază din mulțimea curentă pentru a obține fișierele apărute în captura curentă.

```python
noi = acum - baza
print(len(noi), "SUID noi fata de sistemul curat")
for cale in sorted(noi):
    print(" ", cale)
```

Fig.7.1- Codul sursă (diferenta.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_st
art\laborator_colectii> python -m arsenal.privesc.diferenta
baza: 9  acum: 13
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
```

Fig.7.2-Rezultatul codului (diferenta.py)

Capitolul 8. Clasificarea riscului pentru căile noi
Sarcina: Să construiesc un dicționar nume–risc cu un dictionary comprehension și o listă cu fișierele noi periculoase.

```
risc = {
    nume(cale): ("mare" if nume(cale) in GTFO else "normal")
    for cale in noi
}
periculoase = [cale for cale in sorted(noi) if nume(cale) in GTFO]
print("periculoase NOI:", periculoase)
```

Fig.8.1- Codul sursă (diferenta.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_start\l
aborator_colectii> python -m arsenal.privesc.diferenta
baza: 9  acum: 13
4 SUID noi fata de sistemul curat
  /bin/cp
  /usr/bin/find
  /usr/bin/nmap
  /usr/bin/vim.basic
periculoase NOI: ['/bin/cp', '/usr/bin/find', '/usr/bin/nmap', '/usr/bin/vim.basic
']
```

Fig.8.2-Rezultatul codului (diferenta.py)

Capitolul 9. Testarea diferenței dintre mulțimi
Sarcina: Să verific cu un test simplu că scăderea mulțimilor păstrează numai elementele noi.

```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
    baza = {"a", "b"}
    assert acum - baza == {"c"}
```

Fig.9.1- Codul sursă (test diferenta.py)

```
PS C:\Users\moroz\Documents\Codex\2026-11-10\fa\outputs\Laborator_Colectii_start\l
aborator_colectii> pytest -q arsenal/privesc/test_diferenta.py
.                                                                        [100%]
1 passed in 0.03s
```

Fig.9.2-Rezultatul codului (test_diferenta.py)

CONCLUZII

În lucrare am folosit lista pentru păstrarea liniilor citite, tuplul pentru despachetarea regulilor sudo, mulțimea pentru eliminarea duplicatelor și compararea căilor, iar dicționarul pentru gruparea constatărilor și clasificarea riscului. Raportul JSON reunește constatările SUID, sudo și cron, iar diferența față de baza curată evidențiază cele patru fișiere SUID noi.

BIBLIOGRAFIE

1. Visual Studio Code
2. Windows Powershell